

Coffee Shop Chain — Central Database Design

Relational design and implementation across PostgreSQL, MySQL and IBM Db2

Five disconnected data sources consolidated into one normalized database,
then served out as governed subsets to three different database engines.

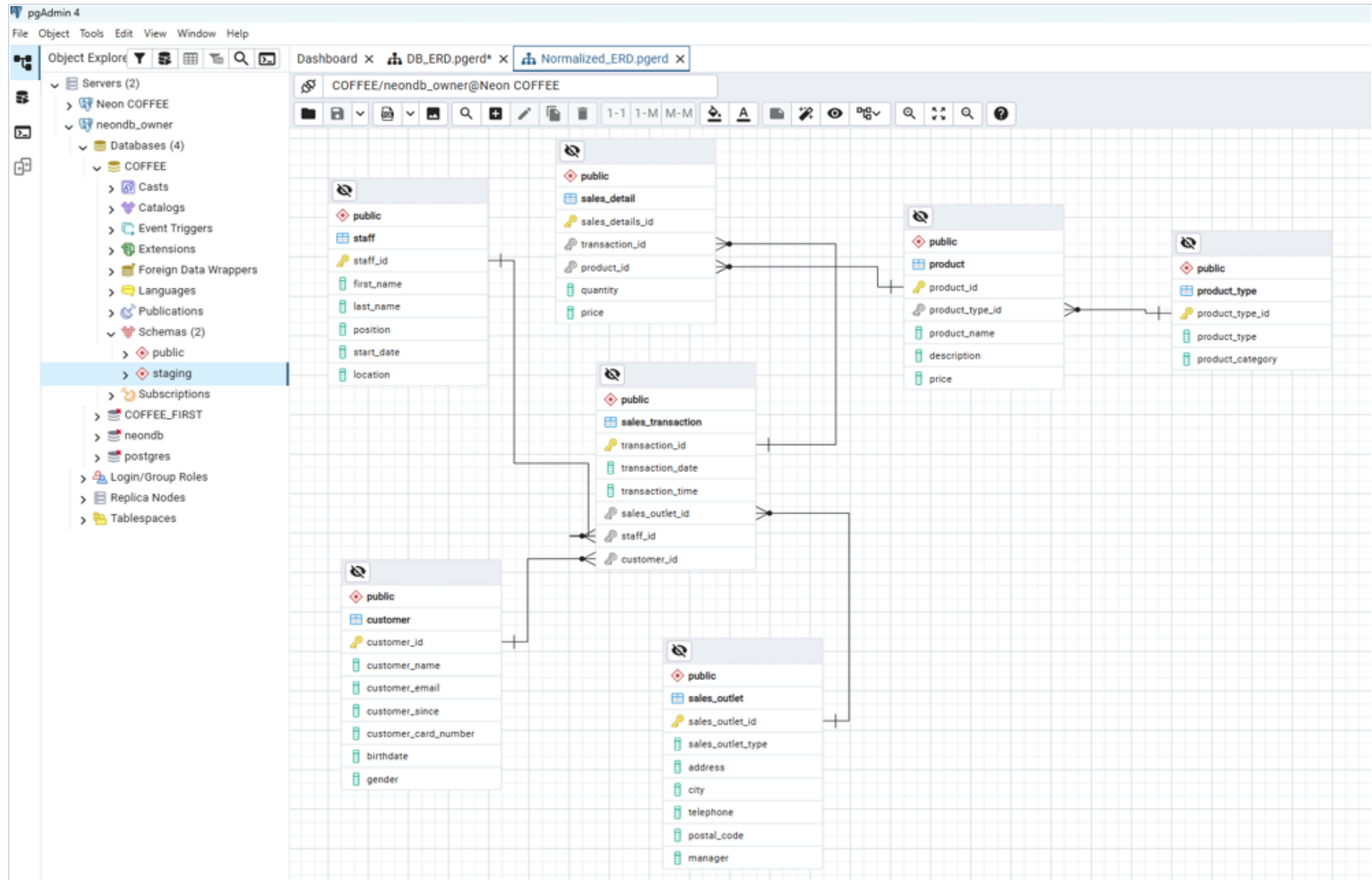
PostgreSQL (Neon) · pgAdmin 4

MySQL (XAMPP) · MySQL Workbench

IBM Db2 on Cloud · Db2 Web Console

1. Entity Relationship Diagram

Final normalized schema in the pgAdmin ERD tool: seven tables, six one-to-many relationships.



2. Schema Definition

DDL executed in pgAdmin against the PostgreSQL database, with named constraints and explicit data types.

The screenshot displays the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure: Servers (2) > Neon COFFEE > neondb_owner > Databases (4) > COFFEE. The 'COFFEE' database is selected, and its 'Schemas (2)' are expanded, showing 'public' and 'staging'. The main pane shows the 'Query' editor with the following SQL code:

```
47
48 -- sales_outlet
49 -- manager holds a staff_id: the employee who runs the outlet.
50 -- Nullable, because the warehouse has no store manager.
51
52 CREATE TABLE public.sales_outlet
53 (
54     sales_outlet_id integer NOT NULL,
55     sales_outlet_type character varying(20),
56     address character varying(100),
57     city character varying(50),
58     telephone character varying(20),
59     postal_code character varying(10),
60     manager integer,
61
62     CONSTRAINT pk_sales_outlet PRIMARY KEY (sales_outlet_id)
63 );
64
65
66 -- customer
67 -- customer_id 0 is reserved for walk-in (non-loyalty) sales.
68
69
70 CREATE TABLE public.customer
71 (
72     customer_id integer NOT NULL,
73     customer_name character varying(80),
74     email character varying(80),
75     reg_date date,
76     card_number character varying(20),
77     date_of_birth date,
78     gender character(1),
79
80     CONSTRAINT pk_customer PRIMARY KEY (customer_id)
```

The 'Query History' pane is empty. The 'Data Output' pane at the bottom shows 'Messages' and 'Notifications'.

Data Output Messages Notifications

No data output. Execute a query.

3. Cloud Database — Neon

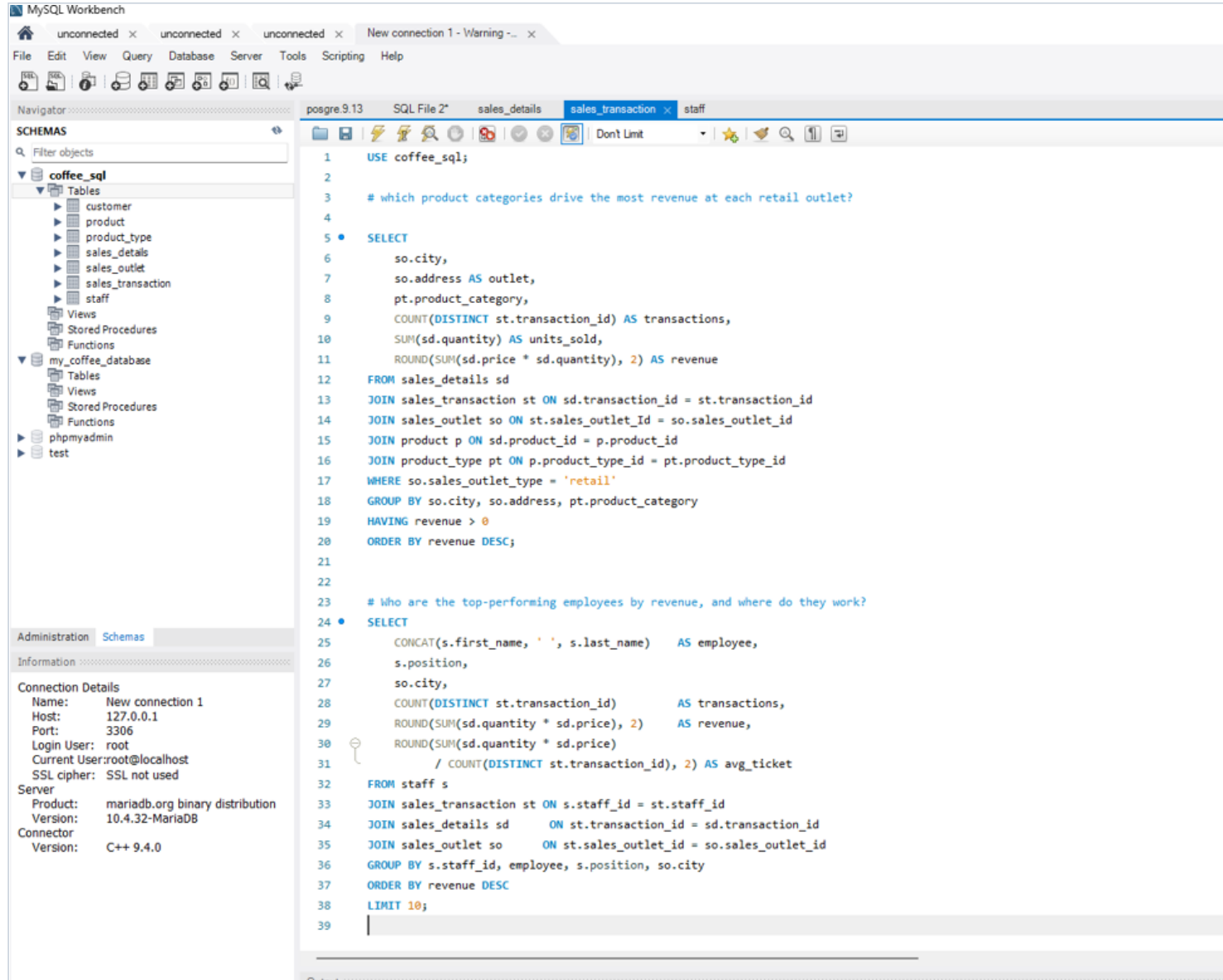
The same schema run through the Neon browser SQL editor. PostgreSQL hosted in the cloud rather than a course sandbox.

The screenshot displays the Neon SQL Editor interface. On the left, a sidebar shows the project structure under 'production', including a 'Connect' button and a list of branches. The main area is titled 'SQL Editor' and shows a list of recent queries, such as 'create staging staff table with employee details' and 'rename database from COFFEE to COFFEE_FIRST'. The right pane shows the SQL code for creating two tables: 'staging.sales_transaction' and 'staging.sales_detail'. The code includes comments and SQL syntax for creating tables with specific columns and constraints. The bottom status bar indicates 'Ready to connect' and features a 'Run' button.

```
110      CONSTRAINT pk_product PRIMARY KEY (product_id)
111    );
112
113
114    -----
115    -- sales_transaction (RECEIPT HEADER)
116    -- One row per sale. transaction_id is unique here because the
117    -- product line items were moved to sales_detail.
118    -----
119    CREATE TABLE staging.sales_transaction
120    (
121      transaction_id integer NOT NULL,
122      transaction_date date,
123      transaction_time time without time zone,
124      sales_outlet_id integer,
125      staff_id integer,
126      customer_id integer,
127
128      CONSTRAINT pk_sales_transaction PRIMARY KEY (transaction_id)
129    );
130
131
132    -----
133    -- sales_detail (RECEIPT LINE ITEMS)
134    -- price is stored here rather than read from product, so the amount
135    -- actually charged survives later catalogue price changes.
136    -----
137    CREATE TABLE staging.sales_detail
138    (
139      sales_detail_id integer NOT NULL,
140      transaction_id integer,
```

4. MySQL — Analytical Queries

Schema rebuilt in MySQL and queried in Workbench: revenue by product category per outlet, and employee performance.



The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a tree view of the database structure, including tables like 'customer', 'product', 'product_type', 'sales_details', 'sales_outlet', 'sales_transaction', and 'staff'. The 'my_coffee_database' is selected. The main editor window shows a SQL query with two parts: one for revenue by product category per outlet, and another for top-performing employees by revenue and location. The query is as follows:

```
1  USE coffee_sql;
2
3  # which product categories drive the most revenue at each retail outlet?
4
5  • SELECT
6      so.city,
7      so.address AS outlet,
8      pt.product_category,
9      COUNT(DISTINCT st.transaction_id) AS transactions,
10     SUM(sd.quantity) AS units_sold,
11     ROUND(SUM(sd.price * sd.quantity), 2) AS revenue
12 FROM sales_details sd
13 JOIN sales_transaction st ON sd.transaction_id = st.transaction_id
14 JOIN sales_outlet so ON st.sales_outlet_id = so.sales_outlet_id
15 JOIN product p ON sd.product_id = p.product_id
16 JOIN product_type pt ON p.product_type_id = pt.product_type_id
17 WHERE so.sales_outlet_type = 'retail'
18 GROUP BY so.city, so.address, pt.product_category
19 HAVING revenue > 0
20 ORDER BY revenue DESC;
21
22
23 # Who are the top-performing employees by revenue, and where do they work?
24 • SELECT
25     CONCAT(s.first_name, ' ', s.last_name) AS employee,
26     s.position,
27     so.city,
28     COUNT(DISTINCT st.transaction_id) AS transactions,
29     ROUND(SUM(sd.quantity * sd.price), 2) AS revenue,
30     ROUND(SUM(sd.quantity * sd.price)
31           / COUNT(DISTINCT st.transaction_id), 2) AS avg_ticket
32 FROM staff s
33 JOIN sales_transaction st ON s.staff_id = st.staff_id
34 JOIN sales_details sd ON st.transaction_id = sd.transaction_id
35 JOIN sales_outlet so ON st.sales_outlet_id = so.sales_outlet_id
36 GROUP BY s.staff_id, employee, s.position, so.city
37 ORDER BY revenue DESC
38 LIMIT 10;
39
```

The bottom left pane shows 'Connection Details' for 'New connection 1', including host '127.0.0.1', port '3306', login user 'root', and current user 'root@localhost'.

5. IBM Db2 — Materialized Query Table

The same reporting objects in Db2. A view, plus an MQT — Db2's equivalent of a materialized view.

The screenshot displays the IBM Db2 on Cloud web interface. On the left, a sidebar shows a tree of data objects under the schema 'TMC83249'. The tree includes Tables (CUSTOMER, PRODUCT, PRODUCT_TYPE, SALES_DETAIL, SALES_OUTLET, SALES_TRANSACTION, STAFF, STAFF_LOCATION_VIEW), Views (CUSTOMER_LOYALTY_VIEW), and MQTs (PRODUCT_INFO_MQT). The main area is divided into a SQL editor and a results pane.

The SQL editor contains the following code:

```
1  -- MQT: Db2's materialized view. Stores data on disk until REFRESH.
2  CREATE TABLE product_info_mqt AS (
3      SELECT
4          p.product_id,
5          p.product_name,
6          p.description,
7          pt.product_type,
8          pt.product_category,
9          p.product_price
10     FROM product p
11     JOIN product_type pt
12        ON p.product_type_id = pt.product_type_id
13 )
14 DATA INITIALLY DEFERRED
15 REFRESH DEFERRED;
16
17 REFRESH TABLE product_info_mqt;
18
19 -- View: stores the query, not the data. Always current.
20 CREATE OR REPLACE VIEW customer_loyalty_view AS
21 SELECT
22     CASE WHEN customer_id = 0 THEN 'Walk-in' ELSE 'Loyalty member' END AS customer_type,
23     COUNT(*) AS transactions
24 FROM sales_transaction
25 GROUP BY
26     CASE WHEN customer_id = 0 THEN 'Walk-in' ELSE 'Loyalty member' END;
```

The results pane shows a table with the following data:

STAFF_ID	FIRST_NAME	LAST_NAME	position
1	Sue	Tindale	CFO
2	Ian	Tindale	CEO